AI Systems Engineer Roadmap

From Python beginner to building production AI SaaS, with daily GitHub commits and real-world projects at every stage.

60 weeks | 8 phases | 20+ shipped projects | 1 commit a day

1. The big picture

Learn in layers, and build something real at the end of every layer. You do not need to become a researcher first. Theory is picked up as you build, not before.

Phase	Weeks	Focus	Flagship project
1	1-6	Python, Git, Linux, SQL, APIs	Expense tracker CLI + GitHub stats tool
2	7-10	Math, NumPy, Pandas, data cleaning	Analysis of a real public dataset
3	11-18	Classical machine learning	Churn predictor served via an API
4	19-26	Deep learning, PyTorch, Transformers	Deployed image + text classifiers
5	27-33	LLMs and RAG	PDF chatbot / University knowledge assistant
6	34-40	AI agents	Research agent + sales agent
7	41-48	Backend, MLOps, evaluation, safety	Production-grade deployed AI service
8	49-60	System design + capstone	Multi-tenant AI SaaS with real users

Time needed: about 2 to 3 hours on weekdays and 4 to 5 hours on one weekend day (roughly 15 to 20 hours a week). If you can study full time, you can compress this to about 8 months. Treat the week numbers as guides, not deadlines.

Priority skills (go deeper here)

- **AI agents** and **system design** are what separate an AI engineer from someone who only calls an LLM API.
- **RAG** is the most requested skill in real AI products today.
- **Backend engineering** is what turns a notebook demo into a product.

2. Your daily GitHub system

The goal is a real, honest streak of learning and building, not green squares for their own sake. Hiring managers read commit history, so make it meaningful.

Set up on day 1

- Create a GitHub account with a professional username and a profile README (a repo named after your username).
- Create one repo called **daily-log**. Add a short entry every day: what you learned, what broke, what you fixed.
- Create one repo **per project**. Never dump everything into one giant repo.
- Set up SSH keys and learn `git add / commit / push / branch / merge / pull request`.

The daily routine (about 2.5 hours)

Time	Activity
30 min	Learn: one lesson, doc, or chapter on today's topic
90 min	Build: apply it in a project or exercise. Write real code.
20 min	Commit and push with a clear message. Update daily-log.
10 min	Plan tomorrow: write the next small task in your TODO file

Commit rules

- Commit small and often. One idea per commit.
- Write clear messages: `feat: add PDF chunking`, `fix: handle empty query`, `docs: add setup steps`.
- Never commit secrets. Keep API keys in a `.env` file and add it to `.gitignore`.
- On a rest or heavy-theory day, commit your notes, a small refactor, tests, or README improvements. That still counts.
- Do not fake commits with empty edits. Real work only; it shows in interviews.

Weekly and monthly rhythm

- **Sunday:** review the week, tidy READMEs, tag a release (v0.1, v0.2...) on any project that moved forward.
- **Monthly:** finish and polish one project completely: README, screenshots or demo GIF, tests, setup instructions.
- **Every project repo needs:** a README with problem, demo, tech stack, how to run it, and what you learned.

> **Streak insurance:** if you miss a day, never miss two. Keep a 10-minute fallback task ready (read code, fix a typo, add a test) so the habit never breaks.

3. The phases

Each phase lists what to learn and the projects to ship. Do not move on until the 'done when' check is true.

Phase 1: Programming foundations

Weeks 1-6 | Be able to build and ship small tools.

What to learn

- Python: syntax, functions, OOP, file I/O, error handling, virtual environments
- Data structures and algorithms (lists, dicts, sets, trees, sorting, searching, Big-O)
- Git and GitHub workflow, Linux command line
- APIs and JSON: call REST APIs with `requests`
- SQL and a database (SQLite, then PostgreSQL)

Real-world projects to push to GitHub

Project	What it does and the real-world angle
Expense tracker CLI	Track spending in SQLite, monthly summaries, export to CSV. Useful for your own life.
GitHub stats tool	Calls the GitHub API and reports your commits, languages and streak. Great for tracking your own roadmap.
Web scraper + alert bot	Scrapes prices or job posts and sends a Telegram or email alert when something changes.

You are done when: you can build a small tool without a tutorial, and your daily-log has 40+ days of entries.

Phase 2: Math and data skills

Weeks 7-10 | Think in vectors and handle messy data.

What to learn

- Linear algebra: vectors, matrices, dot product
- Probability and statistics: distributions, mean/variance, Bayes
- Calculus basics: derivatives, gradients; optimization (gradient descent)
- NumPy, Pandas, Matplotlib; data cleaning and feature engineering
- Train / validation / test splits

Real-world projects to push to GitHub

Project	What it does and the real-world angle
Real dataset analysis	Pick a public dataset (e.g. Sri Lanka tourism, transport, or Kaggle) and write a full EDA report with charts.
Gradient descent from scratch	Implement linear regression with only NumPy to truly understand training.
Data cleaning pipeline	A reusable script that takes messy CSVs and outputs clean data, with tests.

You are done when: you can load, clean, explore, and explain a dataset on your own.

Phase 3: Classical machine learning

Weeks 11-18 | Train, evaluate and ship models.

What to learn

- Supervised vs unsupervised learning, regression, classification, clustering
- Decision trees, Random Forest, Gradient Boosting (XGBoost, LightGBM), SVM, KNN
- Evaluation: precision, recall, F1, ROC-AUC, cross-validation, over/underfitting
- scikit-learn pipelines, hyperparameter tuning, saving models
- Serve a model behind a FastAPI endpoint

Real-world projects to push to GitHub

Project	What it does and the real-world angle
Spam classifier	Train on real SMS/email data, expose it as an API, and add a simple web form.
House-price predictor	Use local or public housing data; compare models and explain feature importance.
Customer churn predictor	End-to-end: data, model, API, Docker container, README with metrics. First portfolio centerpiece.

You are done when: you can take a problem from raw data to a deployed model API and explain your metrics.

Phase 4: Deep learning

Weeks 19-26 | Understand neural networks and Transformers.

What to learn

- Neural networks, backpropagation, activations, loss functions, optimizers
- PyTorch: tensors, autograd, training loops, GPUs (use Colab or Kaggle)
- CNNs for images, RNNs/LSTMs for sequences
- Attention and Transformers; Hugging Face ecosystem
- Fine-tuning basics, transfer learning

Real-world projects to push to GitHub

Project	What it does and the real-world angle
Image classifier app	Fine-tune a CNN on a real problem (plant disease, food, defects) and deploy it as a web demo.
Sentiment analysis service	Fine-tune a Hugging Face model on product or social reviews; expose an API.
Mini Transformer from scratch	Implement attention in PyTorch and train a tiny language model to learn how it works inside.

You are done when: you can train a PyTorch model, fine-tune a pretrained one, and explain attention in your own words.

Phase 5: LLMs and RAG

Weeks 27-33 | Build AI that uses private knowledge.

What to learn

- Tokens, embeddings, context windows, LLM APIs, structured outputs, tool calling
- Prompt design, and when to fine-tune (LoRA/QLoRA) vs when to use RAG
- RAG pipeline: document parsing, chunking, embeddings, vector databases (pgvector, Qdrant, Chroma)
- Hybrid search, metadata filtering, reranking, citations
- Retrieval evaluation: build a test set of questions with known answers

Real-world projects to push to GitHub

Project	What it does and the real-world angle
PDF AI chatbot	Upload PDFs, ask questions, get answers with page citations. Your first RAG system.
University knowledge assistant	Answers student questions from handbooks, timetables and policies. A real client could be a university or school.
RAG customer-support bot	Ingest a company's FAQs and docs, answer with sources, and escalate to a human when unsure. Include an eval report.

You are done when: your RAG app has measured accuracy (not just 'it looks good') and cites its sources.

Phase 6: AI agents

Weeks 34-40 | Systems that plan, use tools and act.

What to learn

- Function calling, agent loops, planning, memory, state management
- Frameworks: LangGraph or similar, plus how to build a simple agent loop without a framework
- Multi-agent patterns, human-in-the-loop approval, tool permissions
- Guardrails, retries, cost and latency limits
- Agent evaluation: success rate, tool-call accuracy, traces

Real-world projects to push to GitHub

Project	What it does and the real-world angle
AI research agent	Takes a topic, searches the web, reads sources, writes a cited report. Add a trace log of every step.
AI sales / lead agent	Reads leads from a CRM or sheet, researches each company, drafts outreach, waits for human approval before sending.
AI coding assistant	Reads a repo, answers questions, proposes patches and runs tests in a sandbox.

You are done when: your agents run reliably on 20+ test tasks, have step limits, and ask for approval before risky actions.

Phase 7: Backend, MLOps and safety

Weeks 41-48 | Turn AI demos into reliable products.

What to learn

- FastAPI, authentication and authorization, async, queues (Celery/Redis), caching
- PostgreSQL, migrations, WebSockets and streaming responses
- Docker, CI/CD with GitHub Actions, cloud deployment (AWS, GCP or Azure)
- Logging, monitoring, cost tracking, experiment tracking (MLflow)
- Safety: prompt injection, data leakage, access control, red-teaming, rate limiting
- Basic frontend with React or Next.js to ship a full product

Real-world projects to push to GitHub

Project	What it does and the real-world angle
Production RAG service	Take your support bot: add login, per-user documents, background ingestion, streaming answers, Docker and CI.
AI document-processing system	Invoices or contracts in, structured data out, with validation, human review queue and audit logs.
Monitoring dashboard	Track latency, token cost, errors and answer quality for your deployed apps.

You are done when: your service is deployed on the internet, has tests and CI, logs every request, and survives bad input.

Phase 8: System design and capstone SaaS

Weeks 49-60 | Design, launch and iterate on a real product.

What to learn

- System design for AI: chatbots, RAG, recommenders, support systems, voice, real-time
- Multi-tenant architecture: data isolation, per-customer config, billing (Stripe)
- Scaling, queues, rate limits, caching, cost control
- Product skills: talk to users, pricing, onboarding, feedback loops

Real-world projects to push to GitHub

Project	What it does and the real-world angle
Multi-tenant AI SaaS (capstone)	Pick one niche (e.g. AI assistant for clinics, schools, law firms or online shops). Combine RAG + agents + database + external APIs.
Find 3 to 5 real users	Offer free access to local businesses or student groups. Collect feedback and fix what hurts.
Case-study write-up	A README and blog post: problem, architecture diagram, metrics, mistakes, results.

You are done when: real people use your product, you have measured results, and you can explain every design decision in an interview.

4. Making projects truly real-world

A tutorial project and a real project can use the same code. The difference is these seven things:

- **Real data:** use messy, real datasets or real documents, not toy examples.
- **Real user:** build for a named person or business, even if it is a friend, a class, or a local shop.
- **Deployed:** a live link beats a screenshot. Free tiers on Render, Railway, Fly.io, Hugging Face Spaces or Vercel are enough.
- **Measured:** report numbers: accuracy, latency, cost per query, time saved.
- **Tested:** unit tests, a small evaluation set, and CI that runs on every push.
- **Documented:** README with architecture diagram, setup steps, demo GIF, and known limitations.
- **Maintained:** fix bugs and ship updates after launch. Real software lives on.

Project repo template

```
project-name/
  README.md          problem, demo, architecture, how to run, results
  src/               application code
  tests/             unit tests and evaluation set
  data/              sample data only (no secrets or private files)
  docs/              diagrams, design notes
  Dockerfile
  .github/workflows/ci.yml
  .env.example       variable names only, never real keys
  .gitignore
```

Ideas for portfolio projects people actually want

- AI assistant for a school or university that answers from official documents
- Invoice or receipt reader that extracts data into a spreadsheet
- WhatsApp or website support bot for a small business
- Resume screener with a human-review step
- Meeting notes and action-item extractor for teams
- Local-language (Sinhala or Tamil) chatbot or translation helper

5. Milestones and checklists

When	You should have
Week 6	40+ commits, 3 Python tools, comfortable with Git and SQL
Week 18	A deployed ML API, a data analysis report, 100+ days of commits
Week 26	PyTorch models trained, one deployed deep-learning demo
Week 33	A working RAG chatbot with an evaluation report
Week 40	Two agents that work reliably with human approval steps
Week 48	A production-style AI service: auth, Docker, CI, monitoring
Week 60	A launched AI SaaS with real users and a case study

Rules to stay on track

- Do not hop between courses. One resource per topic, finish it, then build.
- Use the 70/30 rule: 70 percent building, 30 percent studying.
- When stuck for more than 45 minutes, ask (an AI tutor, docs, community), then write down what you learned.
- Use AI tools to learn faster, but type and understand your own code, or you will not retain it.
- Share progress publicly: short LinkedIn or X posts about what you shipped each month.
- Revisit old projects every few months and upgrade them with what you now know.

Suggested free and paid resources

- Python and CS: CS50P, Automate the Boring Stuff, NeetCode
- ML: Andrew Ng's Machine Learning Specialization, scikit-learn docs
- Deep learning: fast.ai, Andrej Karpathy's 'Neural Networks: Zero to Hero', PyTorch tutorials
- LLMs, RAG and agents: Hugging Face courses, official LLM provider docs, LangGraph docs
- Backend and deployment: FastAPI docs, Docker docs, GitHub Actions docs
- Practice data: Kaggle, Hugging Face Datasets, government open-data portals

> **Start today:** create your GitHub repo `daily-log`, install Python, and make your first commit. Then do it again tomorrow.